

GitHub repository : <https://github.com/Sohail700/AI-Intern-Assignment>

Demo video: <https://github.com/Sohail700/AI-Intern-Assignment/blob/main/README.md>

README: <https://github.com/Sohail700/AI-Intern-Assignment/blob/main/README.md>

INTERVIEW ACCELERATOR

Assignment 3 — Technical Implementation Explanation

Architecture • AI/LLM Approach • Voice • Dynamic Questioning • Evaluation • Technical Decisions

Purpose

This document explains how the submitted Interview Accelerator prototype satisfies the Assignment 3 requirement for a working, personalised interview experience.

Core flow

JD + Resume	Role Analysis	Candidate Analysis	Job Fit	3-Level Interview	Voice + Adaptive Loop	Performance Report
-------------	---------------	--------------------	---------	-------------------	-----------------------	--------------------

Implemented stack

Next.js 15.5.0	React 19.1.1	TypeScript 5.9.2	OpenAI SDK 5.15.0
PDF parsing (pdf-parse)	DOCX parsing (mammoth)	Browser Speech APIs	Optional camera via getUserMedia

1. Architecture

The prototype uses a single Next.js application with a React client for the interview experience and server-side API routes for AI-powered operations. This keeps the prototype simple enough to deploy as one web application while still separating browser interaction from protected LLM calls.

High-level architecture

Layer	Implementation	Responsibility
Presentation	React / Next.js page	Upload/paste screens, role/candidate dashboards, interview screen, transcript, voice controls and final report.
Client state	React state + browser APIs	Stores analysis, current level/question, answer history, per-answer evaluation, progress, transcript and optional camera preview.
Application API	Next.js Route Handlers	Receives structured requests for analysis, answer evaluation, next-question generation and final report generation.
AI services	OpenAI SDK	Produces structured role/candidate analysis, adaptive questions, per-answer evaluation and final performance report.

Request flow

```

Browser / React UI
|
+--> /api/analyze -----> role + candidate analysis
|
+--> /api/next-question -> personalised next question
|
+--> /api/evaluate-answer -> score the submitted answer
|
+--> /api/evaluate -----> final interview report
|
+--> Browser Speech APIs -> interviewer TTS + candidate STT
|
+--> getUserMedia -----> optional camera preview

```

Document inputs are parsed on the server. PDF files are processed with pdf-parse, DOCX files with mammoth, and TXT/MD files as UTF-8 text. The resulting text is combined with any pasted content before analysis.

The prototype intentionally does not require a database for the core experience. Interview state is sufficient for a focused prototype and can later be moved to persistent storage for interview history, progress tracking or multi-job profiles.

2. AI / LLM Approach

The AI layer is organised into four stages so that the system first understands the job, then the candidate, then conducts an adaptive interview, and finally produces an evidence-based performance report. Each LLM call is requested in structured JSON so the UI can consume predictable fields.

Stage A — Understand the Role

The Job Description is transformed into a structured role profile containing:

- Role / Job Title
- Key Responsibilities
- Required Skills and Preferred Skills
- Technical Competencies and Behavioural Competencies
- Experience Expectations
- Important Keywords and Important Concepts
- Key Qualifications or Requirements

Stage B — Understand the Candidate

The resume is analysed against the role to identify:

- Candidate key skills, relevant experience, projects and achievements
- Strengths against the JD
- Missing skills and weak / insufficient areas
- Resume claims that should be probed during the interview
- Preparation areas
- Job Fit score and a simple readiness-oriented label

Stage C — Adaptive interviewer

The next-question prompt receives the role analysis, candidate evidence, recent answer history and per-answer evaluations. The prompt explicitly instructs the model to avoid repetition and to change difficulty based on the last response.

```
Weak answer
  -> targeted clarification
  -> reassess the knowledge gap

Strong answer
  -> increase difficulty
  -> deeper technical / scenario / edge-case follow-up
```

Stage D — Final evaluation

After the interview, the model generates an overall score, seven competency scores, strengths, weaknesses, preparation gaps, readiness and question-level feedback. The report is framed as preparation feedback rather than a hiring decision.

3. Voice Implementation

Voice is treated as a mandatory core capability. The prototype uses browser-native speech APIs to keep the interaction fast and lightweight without requiring a separate audio streaming backend for the prototype.

Step	Component	Implementation	Result
1	AI interviewer	window.speechSynthesis	The current AI-generated question is spoken aloud.
2	Candidate speech	SpeechRecognition / webkitSpeechRecognition	Microphone input is converted to text in the browser.
3	Transcript	React state + textarea	The candidate can review or edit the live transcript.
4	Fallback	Typed answer	The interview can continue even when speech recognition

The question can also be replayed using text-to-speech. For deployed voice testing, Chrome or Edge is recommended because browser speech support varies.

Video bonus

The prototype optionally uses `navigator.mediaDevices.getUserMedia({ video: true })` to display a local camera preview. No facial-expression or emotion detection is used as a primary evaluation signal; answer quality remains the main basis of assessment.

4. Dynamic Questioning Logic

The interviewer is not implemented as a static list of predetermined questions. The current role profile, candidate profile, previous answers and their evaluations are passed into the next-question generation step. This lets the interviewer behave more like a conversation.

Level 1 — Screening

Primary focus: Resume, motivation, basic understanding, role fit, communication, relevant experience and career goals.

Typical interviewer behaviour: Start from an actual project or skill in the resume and connect it to the target role.

Level 2 — Competency

Primary focus: Job-specific competencies, technical understanding, problem solving, behavioural fit, decision-making and practical application.

Typical interviewer behaviour: Ask for technical decisions, trade-offs, implementation constraints and concrete evidence.

Level 3 — Deep-Dive

Primary focus: Technical depth, reasoning, why/how questions, realistic scenarios, counter-questions and claim verification.

Typical interviewer behaviour: Challenge vague or quantified claims, test edge cases and probe assumptions.

Adaptive decision loop

```

Candidate answer
  |
  v
Per-answer evaluation
(score + assessment + strengths + improvements + follow-up intent)
  |
  +---- weak / vague ----> clarify, simplify, probe gap
  |
  +---- strong -----> raise difficulty, add scenario or edge case
  |
  v
Generate next personalised question
  |
  v
Continue until level progression criteria are met

```

The implementation uses three questions per interview level in the guided prototype flow, then advances to the next level. The overall interview therefore progresses from screening to competency to deep-dive while still adapting each next question to the candidate's actual response.

5. Evaluation Methodology

Evaluation happens at two levels: per-answer evaluation during the interview and final evaluation after the interview. This allows the interviewer to adapt in real time while still producing a consolidated report.

Per-answer evaluation

Each submitted answer is evaluated for practical interview quality, including:

- Relevance and directness
- Specificity and concrete evidence
- Reasoning, assumptions and trade-offs
- Technical depth appropriate to the interview level
- Quantification where the question calls for impact or measurement
- A clear follow-up intent that informs the next question

The structured evaluation object is:

```
AnswerEvaluation = { score: 0-100, assessment, strengths[], improvement[], followUpIntent }
```

Final performance report

The report includes the seven competency dimensions requested by the assignment:

- Role Fit
- Technical Knowledge
- Problem Solving
- Communication
- Confidence
- Depth of Understanding
- Behavioural Fit

It also includes question-level feedback with the question, candidate answer, assessment, what was good, what could be better, an ideal direction, and a score. Preparation gaps are ranked by priority, and a readiness label is derived from interview performance, competency scores and JD alignment.

Actionable feedback principle

The product avoids generic feedback such as “improve your communication.” Instead, it aims to identify the missing evidence or reasoning in the answer. For example, when an answer explains implementation but lacks measurable impact, the feedback asks the candidate to add baseline performance, improvement achieved and measurement method.

6. Key Technical Decisions

Next.js + React: One full-stack web framework keeps UI, API routes and deployment simple while remaining maintainable for a prototype.

TypeScript: Shared structured types make the analysis, question, answer and report payloads explicit across the application.

Structured JSON LLM output: JSON response formatting reduces fragile parsing and lets the UI render role analysis, evaluation and reports predictably.

Server-side OpenAI calls: The API key stays on the server rather than being exposed to browser code.

Browser-native voice: SpeechSynthesis and SpeechRecognition provide a credible voice-first interview with minimal infrastructure.

Typed fallback: The interview remains usable when browser microphone support is unavailable or unreliable.

Deterministic demo mode: The prototype can demonstrate the complete product journey without an API key, which makes evaluator testing easier.

Answer-level context: Each answer stores its evaluation so the next question can react to the candidate's actual performance.

No emotion / facial scoring: The assessment prioritises relevance, clarity, evidence, reasoning and depth, matching the assignment's evaluation direction.

No database in MVP: The core flow does not need persistence; database-backed interview history can be added as a production upgrade.

Assignment 3 Requirement Alignment

This implementation is designed around the assignment's required journey: JD/resume input, role understanding, candidate understanding, job fit, three interview levels, adaptive voice interaction, interview evaluation, preparation plan and readiness assessment.

Assignment area	Implemented capability	Where it appears in the prototype
Input	Paste/upload JD and resume	Landing dashboard
Role analysis	Role, skills, responsibilities, competencies, requirements and concepts	Role Analysis screen
Candidate analysis	Skills, experience, projects, achievements, strengths, gaps and claims to probe	Candidate Analysis screen
Job Fit	0–100 preparation-oriented fit score	Candidate Analysis screen
Interview	Screening, Competency and Deep-Dive	Interview screen + level stepper
Adaptivity	Per-answer evaluation drives follow-up intent and next-question difficulty	Interview loop
Voice	Text-to-speech + speech-to-text + typed fallback	Interview controls
Evaluation	Overall score, seven competencies, question feedback	Results screen
Preparation	Strengths, weaknesses and prioritised preparation gaps	Results screen
Readiness	Not Ready / Needs Preparation / Interview Ready / Strong Candidate	Results screen

Design goal: make a candidate feel that performing well here is evidence of genuine interview readiness.